

Agentic Efficiency Report

agentic-crm · zip import · scan SCN-2026-01-01-0001

42

Wasteful + PartialScan

Weighted efficiency verdict

What we found

Overall efficiency score	42 / 100
Verdict	Wasteful (PartialScan)
Files discovered	94
Files parsed	69
Files skipped	25
Tokens analysed	55,038
Estimated monthly token waste	17,534,970
Estimated monthly credit waste	175.35 credits
Estimated monthly dollar waste	\$98.19
Savings range (low / high)	\$78.55 - \$117.83
Duplicate clusters found	2
Oversized context files	3
Overlapping agent groups	1
Review stages inferred	10
Agent-like files detected	46

Category scores

Category	Score	Weight	Penalty	What this means
Redundancy	0/100	25%	-108	2 groups of near-identical files and 4 repeated instruction blocks were found.
Token bloat	33/100	25%	-67	3 context or memory files are over 8,000 tokens.
Review overhead	64/100	20%	-36	10 review or approval steps were inferred from your instructions.
Agent sprawl	61/100	20%	-39	46 agent-style files were found (20 agent roles, 13 skills).
Architecture inefficiency	83/100	10%	-17	9 service directories against 9 hand written source files.

Top 5 waste drivers, in plain language

1. The same instruction block is pasted into 3 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Redundancy · 6,268,000 tokens/month · 62.68 credits · \$35.10

2. Context file is very large: memory.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Token bloat · 2,535,200 tokens/month · 25.35 credits · \$14.20

3. 10 separate review or approval steps were found

Your instructions describe many checks before work is accepted. Each extra check means another pass over the same material, which costs time and money.

Review overhead · 2,045,100 tokens/month · 20.45 credits · \$11.45

4. 46 agent-style files is more than this repo needs

There are many separate agent, skill and prompt files. Each one adds instructions that have to be loaded and kept in sync. Fewer, clearer files cost less and break less often.

Agent sprawl · 1,632,000 tokens/month · 16.32 credits · \$9.14

5. Context file is very large: context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Token bloat · 1,553,200 tokens/month · 15.53 credits · \$8.70

Savings assumptions and formulas

- $\text{estimated_tokens} = \text{ceil}(\text{character_count} / 4)$
- Report credits: 1 credit = 100,000 tokens.
- Vendor billing: \$1.00 = 100 vendor credits; 1 vendor credit = 2,500 input tokens or 500 output tokens (output costs 5x input).
- Derived rates: \$4.00 per 1M input tokens, \$20.00 per 1M output tokens.
- Waste is assumed to be 90% input tokens and 10% output tokens.
- Each agent asset is assumed to be loaded 200 times per month.
- Aggregate savings show a +/-20% range.
- Similarity uses normalised text so whitespace-only and case-only changes are ignored.
- Files under 200 characters are excluded from duplicate detection.
- Scoring runs in two tiers. Tier 1 is the seven specified penalty rules with their hard caps. Tier 2 is severity scaling: for each category we measure the share of the monthly agent context budget that the category wastes, and deduct proportionally up to that category's tier 2 cap, with a category that wastes 50% or more of the budget taking the full deduction. Without tier 2 the capped rules alone could never produce a score below about 72, so the Wasteful and Critical bands would be unreachable. Both tiers are itemised in the score ledger.
- Monthly agent context budget for this repository = 10,933,400 tokens (54,667 agent asset tokens x 200 runs per month).

Setting	Value
Tokens per report credit	100,000
Vendor credits per dollar	100.0
Input tokens per vendor credit	2,500
Output tokens per vendor credit	500
\$ per 1M input tokens	\$4.00
\$ per 1M output tokens	\$20.00
Agent runs per month	200
Output token share of waste	10%
Aggregate variance	+/-20%
Rates last refreshed	never
Rates source	Product owner supplied defaults (not yet refreshed)

How the score was calculated

Penalty rule	Hits	Points each	Cap	Applied	Category
Near-duplicate agent/context file pair (similarity >= 0.80)	20	5	32	-32	redundancy
Repeated instruction block >= 150 chars in 3+ files	4	4	20	-16	redundancy
Context or memory file over 8,000 tokens	3	6	24	-18	token_bloat
Agent-like files above 12	34	2	20	-20	agent_sprawl
Review/approval stages above 4	6	5	15	-15	review_overhead
Microservice mismatch (>= 8 service dirs and < 120 source files)	1	12	12	-12	architecture_inefficiency
Monolith mismatch (1 service and >= 10 overlapping agent roles)	0	10	10	-0	architecture_inefficiency

Penalty rule	Hits	Points each	Cap	Applied	Category
Severity scaling - share of the monthly agent context budget wasted by duplication	80	0	60	-60	redundancy
Severity scaling - share of the monthly agent context budget wasted by oversized files	41	0	60	-49	token_bloat
Severity scaling - share of the monthly agent context budget wasted on review loops	19	0	55	-21	review_overhead
Severity scaling - share of the monthly agent context budget wasted on extra agents	16	0	60	-19	agent_sprawl
Severity scaling - share of the monthly agent context budget lost to the architecture	5	0	50	-5	architecture_inefficiency

Findings (14 issues)

Issue	Severity	Category	Title	Impacted files	Tokens /mo	Credits/mo	\$/mo
ISS-2026-07-29-0006	critical	redundancy	The same instruction block is pasted into 3 files	context/context.md, context/product-context.md, memory/memory.md	6,268,000	62.68	\$35.10
ISS-2026-07-29-0007	high	token_bloat	Context file is very large: memory.md	memory/memory.md	2,535,200	25.35	\$14.20
ISS-2026-07-29-0011	high	review_overhead	10 separate review or approval steps were found	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md	2,045,100	20.45	\$11.45
ISS-2026-07-29-0010	high	agent_sprawl	46 agent-style files is more than this repo needs	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+21 more)	1,632,000	16.32	\$9.14
ISS-2026-07-29-0008	high	token_bloat	Context file is very large: context.md	context/context.md	1,553,200	15.53	\$8.70
ISS-2026-07-29-0001	high	redundancy	20 files are near copies of each other	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+16 more)	1,055,200	10.55	\$5.91
ISS-2026-07-29-0003	medium	redundancy	The same instruction block is pasted into 43 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+39 more)	764,400	7.64	\$4.28
ISS-2026-07-29-0012	high	architecture_inefficiency	Split into many services but there is little code in them	services/svc-1, services/svc-2, services/svc-3, services/svc-4 (+5 more)	546,670	5.47	\$3.06
ISS-2026-07-29-0009	medium	token_bloat	Context file is very large: product-context.md	context/product-context.md	401,000	4.01	\$2.25
ISS-2026-07-29-0004	medium	redundancy	The same instruction block is pasted into 23 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+19 more)	391,600	3.92	\$2.19
ISS-2026-07-29-0005	low	redundancy	The same instruction block is pasted into 20 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+16 more)	182,400	1.82	\$1.02
ISS-2026-07-29-0013	low	agent_sprawl	13 skill files, and some repeat each other	skills/archive.skill.md, skills/classify.skill.md, skills/escalate.skill.md, skills/extract.skill.md (+9 more)	72,000	0.72	\$0.40
ISS-2026-07-29-0014	low	review_overhead	10 orchestration files create extra hand-offs	instruction.md, instructions.md, orchestrator.md, prompts/stage-1.prompt.md (+6 more)	50,000	0.50	\$0.28
ISS-2026-07-29-0002	low	redundancy	2 files are near copies of each other	instructions.md, orchestrator.md	38,200	0.38	\$0.21

Evidence detail

ISS-2026-07-29-0006 — The same instruction block is pasted into 3 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 62678 characters (~15670 tokens) found in: context/context.md, context/product-context.md, memory/memory.md | Sample: "The customer has an established relationship with the platform and has raised several tickets about billing, onboarding and reporting over the last four quarter"

Formula: $\text{monthly_waste} = \text{block_tokens} \times (\text{files_containing_block} - 1) \times \text{runs_per_month}$

Recommendation: Move the shared block into one file and reference it from the others. (impact Medium, effort Small)

ISS-2026-07-29-0007 — Context file is very large: memory.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Evidence: memory/memory.md is about 20,676 tokens which is 12,676 tokens over the 8,000 token guideline.

Formula: $\text{monthly_waste} = (\text{file_tokens} - 8000) \times \text{runs_per_month}$

Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0011 — 10 separate review or approval steps were found

Your instructions describe many checks before work is accepted. Each extra check means another pass over the same material, which costs time and money.

Evidence: Stages inferred: Architecture review, Code review, Escalation review, Final approval, Human in the loop, Peer review, Product approval, Security review, Self review, Sign-off. Examples: Architecture review -> agents/billing.agent.md; Code review -> agents/billing.agent.md; Escalation review -> agents/billing.agent.md; Final approval -> agents/billing.agent.md

Formula: $\text{monthly_waste} = \text{extra_review_stages} \times \text{total_instruction_tokens} \times \text{runs_per_month} \times 0.25$

Recommendation: Keep at most four review gates. Combine the overlapping ones into a single checklist. (impact High, effort Medium)

ISS-2026-07-29-0010 — 46 agent-style files is more than this repo needs

There are many separate agent, skill and prompt files. Each one adds instructions that have to be loaded and kept in sync. Fewer, clearer files cost less and break less often.

Evidence: 46 agent-like files detected against a healthy guideline of 12. Median agent file size is 240 tokens.

Formula: $\text{monthly_waste} = \text{extra_agent_files} \times \text{median_agent_tokens} \times \text{runs_per_month}$

Recommendation: Group agents by job. Merge the ones that overlap and delete the ones nobody calls. (impact High, effort Large)

ISS-2026-07-29-0008 — Context file is very large: context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Evidence: context/context.md is about 15,766 tokens which is 7,766 tokens over the 8,000 token guideline.

Formula: $\text{monthly_waste} = (\text{file_tokens} - 8000) \times \text{runs_per_month}$

Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0001 — 20 files are near copies of each other

These files say almost the same thing. Every time your agents run, the same words get sent to the model more than once. Keeping one source of truth removes the repeat cost.

Evidence: Group DUP-001 - highest similarity 100%. Files: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ...

Formula: $\text{monthly_waste} = (\text{sum of duplicate file tokens} - \text{largest file tokens}) \times \text{runs_per_month}$

Recommendation: Keep the most complete file, delete or shrink the copies, and link to the kept file instead. (impact High, effort Medium)

ISS-2026-07-29-0003 — The same instruction block is pasted into 43 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 363 characters (~91 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "Before you begin any task you must read the repository conventions file, confirm the ticket number, check that the branch name follows the release naming policy"

Formula: $\text{monthly_waste} = \text{block_tokens} \times (\text{files_containing_block} - 1) \times \text{runs_per_month}$

Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0012 — Split into many services but there is little code in them

The repo is arranged as many small services, yet the amount of real code is small. Each service still needs its own setup and its own instructions, so the overhead is larger than the work being done.

Evidence: 9 service directories detected with only 9 hand written source files.

Formula: $\text{monthly_waste} = 5\% \times \text{total_agent_asset_tokens} \times \text{runs_per_month}$

Recommendation: Fold the thin services back together until each one has a clear, separate job. (impact High, effort Large)

ISS-2026-07-29-0009 — Context file is very large: product-context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Evidence: context/product-context.md is about 10,005 tokens which is 2,005 tokens over the 8,000 token guideline.

Formula: $\text{monthly_waste} = (\text{file_tokens} - 8000) \times \text{runs_per_month}$

Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0004 — The same instruction block is pasted into 23 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 353 characters (~89 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "Every change moves through the following gates in order: self review, then peer review, then code review by the owning team, then a security review, then archit"

Formula: $\text{monthly_waste} = \text{block_tokens} \times (\text{files_containing_block} - 1) \times \text{runs_per_month}$

Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0005 — The same instruction block is pasted into 20 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 190 characters (~48 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "When finished, write a summary into the shared memory file and notify the orchestrator so the next stage can start. Include the ticket number, the files touched"

Formula: $\text{monthly_waste} = \text{block_tokens} \times (\text{files_containing_block} - 1) \times \text{runs_per_month}$

Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0013 — 13 skill files, and some repeat each other

Skill files are meant to be small and specific. When there are many of them and they repeat, the model reads the same guidance several times.

Evidence: 13 skill files detected; 0 of them sit inside near-duplicate groups.

Formula: $\text{monthly_waste} = \text{duplicated_skill_tokens} \times \text{runs_per_month} \times 0.5$

Recommendation: Keep one skill file per capability and delete the near copies. (impact Medium, effort Small)

ISS-2026-07-29-0014 — 10 orchestration files create extra hand-offs

There are many files describing how work moves between agents. Every hand-off adds instructions that must be loaded and kept in step with the others.

Evidence: 10 orchestration or prompt files detected, average size 125 tokens.

Formula: $\text{monthly_waste} = (\text{orchestration_files} - 6) \times \text{avg_orchestration_tokens} \times \text{runs_per_month} \times 0.5$

Recommendation: Describe the flow once in a single orchestrator file and let the agents stay simple. (impact Medium, effort Medium)

ISS-2026-07-29-0002 — 2 files are near copies of each other

These files say almost the same thing. Every time your agents run, the same words get sent to the model more than once. Keeping one source of truth removes the repeat cost.

Evidence: Group DUP-002 - highest similarity 84%. Files: instructions.md, orchestrator.md

Formula: $\text{monthly_waste} = (\text{sum of duplicate file tokens} - \text{largest file tokens}) \times \text{runs_per_month}$

Recommendation: Keep the most complete file, delete or shrink the copies, and link to the kept file instead. (impact Medium, effort Small)

Recommended actions, ranked by impact then effort

#	Action	Category	Impact	Effort	Tokens/mo saved	\$/mo saved
1	Move the shared block into one file and reference it from the others.	Redundancy	High	Small	764,400	\$4.28

#	Action	Category	Impact	Effort	Tokens/mo saved	\$/mo saved
2	Keep at most four review gates. Combine the overlapping ones into a single checklist.	Review overhead	High	Medium	2,045,100	\$11.45
3	Keep the most complete file, delete or shrink the copies, and link to the kept file instead.	Redundancy	High	Medium	1,055,200	\$5.91
4	Group agents by job. Merge the ones that overlap and delete the ones nobody calls.	Agent sprawl	High	Large	1,632,000	\$9.14
5	Fold the thin services back together until each one has a clear, separate job.	Architecture inefficiency	High	Large	546,670	\$3.06
6	Keep one skill file per capability and delete the near copies.	Agent sprawl	Medium	Small	72,000	\$0.40
7	Split this file into a short always-on summary plus detail files that load only when needed.	Token bloat	Medium	Medium	2,535,200	\$14.20
8	Describe the flow once in a single orchestrator file and let the agents stay simple.	Review overhead	Medium	Medium	50,000	\$0.28

File inventory by category

Group	Files	Parsed	Skipped	Tokens
Agents	20	20	0	5,564
Skills	13	13	0	1,403
Context and memory	3	3	0	46,447
Prompt and orchestration	10	10	0	1,253
Source code	9	9	0	90
Diagrams	1	1	0	13
Docs	2	2	0	207
Other text assets	36	11	25	61

Largest 94 files by estimated tokens

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
memory/memory.md	context_memory	Scanned	8	80.8	20,676	-
context/context.md	context_memory	Scanned	7	61.6	15,766	-
context/product-context.md	context_memory	Scanned	5	39.1	10,005	-
agents/legacy/intake-backup.agent.md	agent	Scanned	23	1.1	288	DUP-001
agents/intake.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/legacy/intake-copy.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/legacy/intake-old.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/escalation.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/legacy/intake-v2.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/onboarding.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/forecast.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/proposal.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/reporting.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/billing.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/handoff.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/notes.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/renewal.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/research.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/support.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/legal.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/pricing.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/triage.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/qa.agent.md	agent	Scanned	23	1.1	275	DUP-001
orchestrator.md	orchestration	Scanned	15	0.8	206	DUP-002
instructions.md	orchestration	Scanned	7	0.7	191	DUP-002
prompts/stage-1.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-2.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-3.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-4.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-5.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-6.prompt.md	orchestration	Scanned	5	0.4	109	-
skills/reconcile.skill.md	skill	Scanned	5	0.4	109	-
skills/summarise.skill.md	skill	Scanned	5	0.4	109	-
skills/translate.skill.md	skill	Scanned	5	0.4	109	-
skills/archive.skill.md	skill	Scanned	5	0.4	108	-
skills/classify.skill.md	skill	Scanned	5	0.4	108	-
skills/escalate.skill.md	skill	Scanned	5	0.4	108	-

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
skills/extract.skill.md	skill	Scanned	5	0.4	108	-
skills/schedule.skill.md	skill	Scanned	5	0.4	108	-
skills/validate.skill.md	skill	Scanned	5	0.4	108	-
skills/format.skill.md	skill	Scanned	5	0.4	107	-
skills/legacy/search-old.skill.md	skill	Scanned	5	0.4	107	-
skills/notify.skill.md	skill	Scanned	5	0.4	107	-
skills/search.skill.md	skill	Scanned	5	0.4	107	-
instruction.md	orchestration	Scanned	5	0.4	105	-
README.md	other	Scanned	4	0.4	105	-
docs/guide.md	other	Scanned	4	0.4	102	-
workflow.md	orchestration	Scanned	5	0.4	97	-
config/settings.yaml	other	Scanned	5	0.1	15	-
docs/architecture.mmd	diagram	Scanned	3	0.0	13	-
package.json	other	Scanned	1	0.0	10	-
services/svc-1/index.js	source	Scanned	1	0.0	10	-
services/svc-2/index.js	source	Scanned	1	0.0	10	-
services/svc-3/index.js	source	Scanned	1	0.0	10	-
services/svc-4/index.js	source	Scanned	1	0.0	10	-
services/svc-5/index.js	source	Scanned	1	0.0	10	-
services/svc-6/index.js	source	Scanned	1	0.0	10	-
services/svc-7/index.js	source	Scanned	1	0.0	10	-
services/svc-8/index.js	source	Scanned	1	0.0	10	-
services/svc-9/index.js	source	Scanned	1	0.0	10	-
services/svc-1/package.json	other	Scanned	1	0.0	4	-
services/svc-2/package.json	other	Scanned	1	0.0	4	-
services/svc-3/package.json	other	Scanned	1	0.0	4	-
services/svc-4/package.json	other	Scanned	1	0.0	4	-
services/svc-5/package.json	other	Scanned	1	0.0	4	-
services/svc-6/package.json	other	Scanned	1	0.0	4	-
services/svc-7/package.json	other	Scanned	1	0.0	4	-
services/svc-8/package.json	other	Scanned	1	0.0	4	-
services/svc-9/package.json	other	Scanned	1	0.0	4	-
assets/blob.json	other	Binary	0	0.0	0	-
data-dump.json	other	SkippedOversized	0	6766.5	0	-
assets/asset-0.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-1.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-10.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-11.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-2.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-3.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-4.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-5.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-6.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
assets/asset-7.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-8.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-9.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/logo.svg	other	SkippedUnsupported	0	0.0	0	-
assets/notes.docx	other	SkippedUnsupported	0	0.0	0	-
services/svc-1/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-2/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-3/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-4/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-5/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-6/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-7/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-8/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-9/Dockerfile	other	SkippedUnsupported	0	0.0	0	-

Skipped files and warnings

Skipped by reason: Binary = 1, SkippedOversized = 1, SkippedUnsupported = 23

Path	Status	Size (KB)	Reason
assets/blob.json	Binary	0.0	File contains binary data
data-dump.json	SkippedOversized	6766.5	File is 6.6 MB which is above the 5 MB parse limit
assets/asset-0.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-1.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-10.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-11.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-2.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-3.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-4.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-5.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-6.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-7.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-8.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-9.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list

Path	Status	Size (KB)	Reason
assets/logo.svg	SkippedUnsupported	0.0	Extension '.svg' is not in the supported analysis list
assets/notes.docx	SkippedUnsupported	0.0	Extension '.docx' is not in the supported analysis list
services/svc-1/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-2/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-3/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-4/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-5/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-6/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-7/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-8/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-9/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list

Draft replacement files

Each draft is a refined version of a file that already exists in your repository. Filenames carry an -optimised suffix so nothing is overwritten.

memory-optimised.md (from memory/memory.md, 20,676 -> 860 tokens, -95.8%)

```
# Long Term Memory

## Purpose
Always-on core facts about the customer account. Detailed ticket notes live in on-demand files and should only be loaded when directly relevant to the current task.

## Core (always load)

### Account summary
- The customer has an established relationship with the platform.
- Several tickets have been raised over the last four quarters.
- Ticket topics span three areas: billing, onboarding, and reporting.
- Each ticket was resolved by a different agent.
- Resolution notes from those agents were copied into this memory file.

### Key facts to keep in context
- Relationship status: established, ongoing.
- Time window covered: the last four quarters (rolling).
- Topic areas of record: billing, onboarding, reporting.
- Handling pattern: multi-agent, one agent per ticket.
- Note provenance: copied from each resolving agent's ticket notes.

### How to use this memory
- Treat the above as the always-on core context for any customer interaction.
- Do not restate these facts back to the user unless asked.
- If a task touches billing, onboarding, or reporting, load the matching detail file below before responding.
- If a task does not touch these topics, do not load any detail file.
- Never invent ticket details that are not present in the detail files.

## Detail sections (load on demand)

The original memory file contained only one summary sentence repeated many times, with no distinct ticket details. The structure below is where real per-ticket notes should live. Load a file only when its topic is in scope.

### `memory/history/billing.md`
- Full billing ticket notes and resolutions.
- One entry per ticket with: date, quarter, resolving agent, issue, resolution, follow-up.
- Load when the current task involves invoices, charges, refunds, plans, or payment methods.

### `memory/history/onboarding.md`
- Full onboarding ticket notes and resolutions.
- One entry per ticket with: date, quarter, resolving agent, issue, resolution, follow-up.
- Load when the current task involves setup, provisioning, user creation, or first-run configuration.

### `memory/history/reporting.md`
- Full reporting ticket notes and resolutions.
- One entry per ticket with: date, quarter, resolving agent, issue, resolution, follow-up.
- Load when the current task involves dashboards, exports, metrics, or scheduled reports.

### `memory/history/index.md`
- Chronological index of every ticket across all topics.
- Columns: ticket id, quarter, date, topic, resolving agent, outcome, link to detail file.
- Load when the task needs cross-topic history or trend analysis across quarters.

## Maintenance rules
- Keep this file short. Only the Core section should grow, and only with durable facts.
- When a new ticket is resolved, append its notes to the matching detail file, not here.
- If a fact is repeated across detail files, keep it once and reference it from the others.
- Remove any filler prose. Prefer bullet lists over paragraphs.
- Review quarterly: drop entries older than four quarters from the detail files or archive them under `memory/history/archive/`.

## Acceptance criteria for this memory
- Core section fits on one screen.
- No duplicated sentences or paragraphs.
- Every detail file path listed above is real or created before first use.
- Agents can answer general account questions from Core alone.
- Agents load a detail file only when its topic is in scope for the current task.
```

context-optimised.md (from context/context.md, 15,766 -> 1,093 tokens, -93.1%)

```
# Working Context

## Purpose
```

Always-on core context for agents working in the agentic-crm repo. Detailed history lives in on-demand files listed at the end. Load only what the current task needs.

Pre-task checklist (always on)

Before starting any task, every agent must:

- Read the repository conventions file.
- Confirm the ticket number for the work.
- Check that the branch name follows the release naming policy.
- Make sure the change log entry has been drafted.
- Never commit directly to the `main` branch.
- Run the unit test suite before handoff.
- Run the lint task before handoff.
- Hand work over to the next agent in the chain only after tests and lint pass.

These steps are non-negotiable and apply to every ticket, no matter how small.

Handoff rules (always on)

- Work moves through a chain of agents. Each agent is responsible for leaving the repo in a clean, testable state for the next one.
- Do not skip the test or lint step, even if the change looks trivial.
- If a step fails, fix it or clearly flag it in the handoff notes. Do not pass broken work forward.
- Record the ticket number, branch name, and change log entry status in your handoff notes.

Branch and release policy (always on)

- Branch names must follow the release naming policy defined in the repository conventions file.
- Direct commits to `main` are forbidden. All changes go through a branch and review.
- The change log entry must be drafted as part of the change, not after merge.

Account summary (always on)

Short version of the customer background that agents need at a glance:

- The customer has an established relationship with the platform.
- Over the last four quarters they have raised several support tickets.
- Ticket topics cluster into three areas: billing, onboarding, and reporting.
- Each ticket was resolved by a different agent.
- Resolution notes from each agent were copied into the account history file.
- Because ownership rotated, context is fragmented. Check the detail file before assuming prior state.

Use this summary for routine work. Only load the full history when the current task clearly touches prior tickets.

When to load more context

Load a detail file only if the task requires it. Signals that you need more context:

- The ticket references a past incident, past resolution, or a specific prior agent.
- The work touches billing flows, onboarding flows, or reporting flows for this customer.
- You need to reconcile conflicting notes between agents.
- A stakeholder asks for a history summary or timeline.

If none of these apply, the always-on summary above is enough.

Detail sections (on-demand files)

Move the following into separate files. Reference them by path and load only when needed:

- `context/account-history/billing.md`

Full notes on billing tickets across the last four quarters, including the agent who resolved each one and the outcome.

- `context/account-history/onboarding.md`

Full notes on onboarding tickets, including setup issues, resolution steps, and the agent who closed each ticket.

- `context/account-history/reporting.md`

Full notes on reporting tickets, including report types involved, fixes applied, and the agent who resolved each one.

- `context/account-history/index.md`

A short cross-topic timeline that links to the three files above in ticket order. Use this when you need a chronological view rather than a topic view.

Rules for the detail files

When you split content into the on-demand files above, keep these rules:

- Preserve the original facts: established customer, four quarters of tickets, three topics (billing, onboarding, reporting), different resolving agent per ticket, notes copied from those agents.
- Do not restate the same paragraph many times. One clear entry per ticket is enough.
- Keep each detail file focused on its single topic so agents can load just the slice they need.
- Cross-link between files when a ticket spans more than one topic.

Quick reference

- Conventions: repository conventions file (read first).
- Tests: unit test suite (must pass before handoff).
- Quality: lint task (must pass before handoff).
- Branch policy: release naming policy, no direct commits to `main`.
- Change log: entry drafted as part of the change.

- Customer state: see always-on summary above; load ``context/account-history/*`` only on demand.